Hotel Management System

Algorithms and Data Structures 1 — Project Report

Course Project 2026–2027

---

1 Introduction and Architecture Overview

The objective of this project is to implement a robust management system for a hotel chain containing multiple locations (sites) across different cities, with each location accommodating multiple clients (bookings). All data items and their hierarchical relationships are modeled using **Singly Linked Lists (SLL)**.

The core architecture relies on a three-tier nested structure defined in `tp3.h`:

- `T_Hotel`: Represents the root hotel chain entity, containing a name string and a pointer to the head of the linked list of sites (`T_Site`).
- `T_Site`: Represents an individual hotel location in a specific city, tracking the client count, a pointer to the head of its client list, and a pointer to the next site.
- `T_Client`: Represents an individual customer booking with fields for last name, first name, age, room number, and a pointer to the next client node.

2 Additional Structures and Design Choices

To ensure optimal performance, memory safety, and maintainability beyond the basic project requirements, several auxiliary functions and design choices were introduced:

2.1 Search and Sorting Strategy (`searchClients`)

- **Design Choice:** Rather than creating a separate, dedicated linked list structure to store search results (which would introduce redundant pointer overhead and complex memory management), we implemented a **dynamic array of pointers** (`T_Client **results`) coupled with the C standard library's `qsort` function.
- **Rationale:** A dynamic array allows efficient $O(1)$ amortized collection of matching client references across all sites. Sorting the filtered subset using `qsort` and a custom static comparator function (`compareClients`) cleanly sorts the results alphabetically by last name and then first name without altering the underlying primary data structure.

2.2 Robust Memory Management (`freeHotel`, `freeSite`, `freeClient`)

- **Design Choice:** Comprehensive recursive deallocation functions were implemented to prevent memory leaks.

- **Rationale:** Because nodes are dynamically allocated via `malloc` and strings are duplicated using `strdup`, deleting a site or quitting the program requires descending down the hierarchy (freeing all clients within a site, then the site's city string, then the site itself, and finally the hotel name and hotel structure).

3 Time Complexity Analysis

Below is the algorithmic time complexity analysis for each implemented function in the project:

Function Name	Time Complexity	Explanation / Operational Analysis
`createClient`	$O(1)$	Allocates memory and duplicates strings in constant time.
`createSite`	$O(1)$	Allocates memory for a site and initializes pointers in constant time.
`createHotel`	$O(1)$	Allocates memory for the hotel chain structure in constant time.
`addClient`	$O(N)$	Traverses the site's client list to insert the client in alphabetical order (N = clients at site).
`addSite`	$O(M)$	Traverses the hotel's site list to insert the city in alphabetical order (M = total sites).
`displayHotel`	$O(M)$	Linear traversal over all M sites in the hotel chain to print city statistics.
`displaySite`	$O(N)$	Linear traversal over all N clients at a specific site to print booking details.
`deleteClient`	$O(N)$	Traverses up to N clients at a site to locate, unlink, and free the target booking.
`deleteSite`	$O(M + N_{\text{total}})$	Traverses site list ($O(M)$) and frees all associated client bookings ($O(N_{\text{total}})$).
`searchClients`	$O(M \cdot N_{\text{avg}} + K \log K)$	Traverses all sites/clients to find K matches, then sorts the dynamic array of size K using `qsort`.
`processingfile`	$O(L \cdot (M + N))$	Reads L lines from file, searching/adding sites ($O(M)$) and inserting clients ($O(N)$) per line.
`freeHotel`	$O(M + N_{\text{total}})$	Recursively traverses every node across all sites and clients to release all heap memory.

Table 1: Algorithmic Complexity Summary

4 Conclusion

The implemented Hotel Chain Management System successfully satisfies all structural and functional constraints using singly linked lists in C. Through modular design, strict memory management, and efficient searching algorithms, the program ensures reliability, scalability, and ease of maintenance for administrative hotel tasks.